

1804.10694v5: table evidence

Tables

Identifier	Page	Conf.	LaTeX source	Rendered	Image																																																																																																																																																																																				
1804.10694v5_TAB_001	3	■ 1.000	<pre>\begin{tabular}{t}{ l l l l l l } \hline Feature & Tiramisu & AlphaZ & PENCIL & Pluto & Halide \\ \hline CPU code generation & Yes & Yes & Yes & Yes & Yes \\ \hline GPU code generation & Yes & No & Yes & Yes & Yes \\ \hline Distributed CPU code generation & Yes & No & No & Yes & Yes \\ \hline Distributed GPU code generation & Yes & No & No & No & No \\ \hline Support all affine loop transformations & Yes & Yes & Yes & Yes & No \\ \hline Commands for loop transformations & Yes & Yes & No & No & Yes \\ \hline Commands for optimizing data accesses & Yes & Yes & No & No & Yes \\ \hline Commands for communication & Yes & No & No & No & No \\ \hline Commands for memory hierarchies & Yes & No & No & No & Limited \\ \hline Expressing cyclic data-flow graphs & Yes & Yes & Yes & Yes & No \\ \hline Non-rectangular iteration spaces & Yes & Yes & Yes & Yes & Limited \\ \hline Exact dependence analysis & Yes & Yes & Yes & Yes & No \\ \hline Compile-time set emptiness check & Yes & Yes & Yes & Yes & No \\ \hline Implement parametric tiling & No & Yes & No & No & Yes \\ \hline \end{tabular}</pre>	<table><tr><th>Feature</th><th>Tiramisu</th><th>AlphaZ</th><th>PENCIL</th><th>Pluto</th><th>Halide</th></tr><tr><td>CPU code generation</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td></tr><tr><td>GPU code generation</td><td>Yes</td><td>No</td><td>Yes</td><td>Yes</td><td>Yes</td></tr><tr><td>Distributed CPU code generation</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td><td>Yes</td></tr><tr><td>Distributed GPU code generation</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>No</td></tr><tr><td>Support all affine loop transformations</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Commands for loop transformations</td><td>Yes</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr><tr><td>Commands for optimizing data accesses</td><td>Yes</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr><tr><td>Commands for communication</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>No</td></tr><tr><td>Commands for memory hierarchies</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>Limited</td></tr><tr><td>Expressing cyclic data-flow graphs</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Non-rectangular iteration spaces</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Limited</td></tr><tr><td>Exact dependence analysis</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Compile-time set emptiness check</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Implement parametric tiling</td><td>No</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr></table>	Feature	Tiramisu	AlphaZ	PENCIL	Pluto	Halide	CPU code generation	Yes	Yes	Yes	Yes	Yes	GPU code generation	Yes	No	Yes	Yes	Yes	Distributed CPU code generation	Yes	No	No	Yes	Yes	Distributed GPU code generation	Yes	No	No	No	No	Support all affine loop transformations	Yes	Yes	Yes	Yes	No	Commands for loop transformations	Yes	Yes	No	No	Yes	Commands for optimizing data accesses	Yes	Yes	No	No	Yes	Commands for communication	Yes	No	No	No	No	Commands for memory hierarchies	Yes	No	No	No	Limited	Expressing cyclic data-flow graphs	Yes	Yes	Yes	Yes	No	Non-rectangular iteration spaces	Yes	Yes	Yes	Yes	Limited	Exact dependence analysis	Yes	Yes	Yes	Yes	No	Compile-time set emptiness check	Yes	Yes	Yes	Yes	No	Implement parametric tiling	No	Yes	No	No	Yes	<table><tr><th>Feature</th><th>Tiramisu</th><th>AlphaZ</th><th>PENCIL</th><th>Pluto</th><th>Halide</th></tr><tr><td>CPU code generation</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td></tr><tr><td>GPU code generation</td><td>Yes</td><td>No</td><td>Yes</td><td>Yes</td><td>Yes</td></tr><tr><td>Distributed CPU code generation</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td><td>Yes</td></tr><tr><td>Distributed GPU code generation</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>No</td></tr><tr><td>Support all affine loop transformations</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Commands for loop transformations</td><td>Yes</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr><tr><td>Commands for optimizing data accesses</td><td>Yes</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr><tr><td>Commands for communication</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>No</td></tr><tr><td>Commands for memory hierarchies</td><td>Yes</td><td>No</td><td>No</td><td>No</td><td>Limited</td></tr><tr><td>Expressing cyclic data-flow graphs</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Non-rectangular iteration spaces</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Limited</td></tr><tr><td>Exact dependence analysis</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Compile-time set emptiness check</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr><tr><td>Implement parametric tiling</td><td>No</td><td>Yes</td><td>No</td><td>No</td><td>Yes</td></tr></table>	Feature	Tiramisu	AlphaZ	PENCIL	Pluto	Halide	CPU code generation	Yes	Yes	Yes	Yes	Yes	GPU code generation	Yes	No	Yes	Yes	Yes	Distributed CPU code generation	Yes	No	No	Yes	Yes	Distributed GPU code generation	Yes	No	No	No	No	Support all affine loop transformations	Yes	Yes	Yes	Yes	No	Commands for loop transformations	Yes	Yes	No	No	Yes	Commands for optimizing data accesses	Yes	Yes	No	No	Yes	Commands for communication	Yes	No	No	No	No	Commands for memory hierarchies	Yes	No	No	No	Limited	Expressing cyclic data-flow graphs	Yes	Yes	Yes	Yes	No	Non-rectangular iteration spaces	Yes	Yes	Yes	Yes	Limited	Exact dependence analysis	Yes	Yes	Yes	Yes	No	Compile-time set emptiness check	Yes	Yes	Yes	Yes	No	Implement parametric tiling	No	Yes	No	No	Yes
Feature	Tiramisu	AlphaZ	PENCIL	Pluto	Halide																																																																																																																																																																																				
CPU code generation	Yes	Yes	Yes	Yes	Yes																																																																																																																																																																																				
GPU code generation	Yes	No	Yes	Yes	Yes																																																																																																																																																																																				
Distributed CPU code generation	Yes	No	No	Yes	Yes																																																																																																																																																																																				
Distributed GPU code generation	Yes	No	No	No	No																																																																																																																																																																																				
Support all affine loop transformations	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Commands for loop transformations	Yes	Yes	No	No	Yes																																																																																																																																																																																				
Commands for optimizing data accesses	Yes	Yes	No	No	Yes																																																																																																																																																																																				
Commands for communication	Yes	No	No	No	No																																																																																																																																																																																				
Commands for memory hierarchies	Yes	No	No	No	Limited																																																																																																																																																																																				
Expressing cyclic data-flow graphs	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Non-rectangular iteration spaces	Yes	Yes	Yes	Yes	Limited																																																																																																																																																																																				
Exact dependence analysis	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Compile-time set emptiness check	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Implement parametric tiling	No	Yes	No	No	Yes																																																																																																																																																																																				
Feature	Tiramisu	AlphaZ	PENCIL	Pluto	Halide																																																																																																																																																																																				
CPU code generation	Yes	Yes	Yes	Yes	Yes																																																																																																																																																																																				
GPU code generation	Yes	No	Yes	Yes	Yes																																																																																																																																																																																				
Distributed CPU code generation	Yes	No	No	Yes	Yes																																																																																																																																																																																				
Distributed GPU code generation	Yes	No	No	No	No																																																																																																																																																																																				
Support all affine loop transformations	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Commands for loop transformations	Yes	Yes	No	No	Yes																																																																																																																																																																																				
Commands for optimizing data accesses	Yes	Yes	No	No	Yes																																																																																																																																																																																				
Commands for communication	Yes	No	No	No	No																																																																																																																																																																																				
Commands for memory hierarchies	Yes	No	No	No	Limited																																																																																																																																																																																				
Expressing cyclic data-flow graphs	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Non-rectangular iteration spaces	Yes	Yes	Yes	Yes	Limited																																																																																																																																																																																				
Exact dependence analysis	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Compile-time set emptiness check	Yes	Yes	Yes	Yes	No																																																																																																																																																																																				
Implement parametric tiling	No	Yes	No	No	Yes																																																																																																																																																																																				

Identifier	Page	Conf.	LaTeX source	Rendered	Image																																				
1804.10694v5_TAB_002	4	0.600	<pre>\begin{tabular}[t]{l l } \hline \multicolumn{2}{ c }{We assume that $\mathbf{C}$ and $\mathbf{P}$ are computations, $\mathbf{b}$ is a buffer i and j are loop iterators} \\ \hline \multicolumn{2}{ c }{Commands for loop nest transformations} \\ \hline Command & Description \\ \hline C.tile(i,j,t1,t2,i0,j0,i1,j1) & Tile the loop levels (i,j) of the computation C by $t1 \times t2$. The names of the new loop levels are (i0,j0,i1,j1) where i0 is the outermost loop level and j1 is the innermost. \\ \hline C.interchange(i,j) & Interchange the i and j loop levels of C. \\ \hline C.shift(i,s) & Loop shifting (shift the loop level i by s iterations). \\ \hline C.split(i,s,i0,i1) & Split the loop level i by s. (i0,i1) are the new loop levels. \\ \hline P.compute_at(C,j) & Compute the computation P in the loop nest of C at loop level j. This might introduce redundant computations. \\ \hline C.unroll(i,v) & Unroll the loop level i by a factor v. \\ \hline C.after(B,i) & Indicate that C should be ordered after B at the loop level i (they have the same order in all the loop levels above i). \\ \hline C.inline() & Inline C in all of its consumers. \\ \hline C.set_schedule() & Transform the iteration domain of C using an affine relation (a map to transform Layer I to II expressed in the ISL syntax). \\ \hline \multicolumn{2}{ c }{Commands for mapping loop levels to hardware} \\ \hline C.parallelize(i) & Parallelize the i loop level for execution on a shared memory system. \\ \hline C.vectorize(i,v) & Vectorize the loop level i by a vector size v. \\ \hline C.gpu(i0,i1,i2,i3) & Mark the loop levels i0,i1,i2 and i3 to be executed on GPU. (i0,i1) are mapped to block IDs and (i2,i3) to thread IDs. \\ \hline C.tile_gpu(i0,i1,t1,t2) & Tile the loops i0 and i1 by $t1 \times t2$ and map them to GPU. \\ \hline C.distribute(i) & Parallelize the loop level i for execution on a distributed memory system. \\ \hline \end{tabular}</pre>	(not rendered)	<table><tr><th colspan="2">We assume that C and P are computations, b is a buffer i and j are loop iterators</th></tr><tr><th colspan="2">Commands for loop nest transformations</th></tr><tr><th>Command</th><th>Description</th></tr><tr><td>C.tile(i,j,t1,t2,i0,j0,i1,j1)</td><td>Tile the loop levels (i,j) of the computation C by $t1 \times t2$. The names of the new loop levels are (i0,j0,i1,j1) where i0 is the outermost loop level and j1 is the innermost.</td></tr><tr><td>C.interchange(i,j)</td><td>Interchange the i and j loop levels of C.</td></tr><tr><td>C.shift(i,s)</td><td>Loop shifting (shift the loop level i by s iterations).</td></tr><tr><td>C.split(i,s,i0,i1)</td><td>Split the loop level i by s. (i0,i1) are the new loop levels.</td></tr><tr><td>P.compute_at(C,j)</td><td>Compute the computation P in the loop nest of C at loop level j. This might introduce redundant computations.</td></tr><tr><td>C.unroll(i,v)</td><td>Unroll the loop level i by a factor v.</td></tr><tr><td>C.after(B,i)</td><td>Indicate that C should be ordered after B at the loop level i (they have the same order in all the loop levels above i).</td></tr><tr><td>C.inline()</td><td>Inline C in all of its consumers.</td></tr><tr><td>C.set_schedule()</td><td>Transform the iteration domain of C using an affine relation (a map to transform Layer I to II expressed in the ISL syntax).</td></tr><tr><th colspan="2">Commands for mapping loop levels to hardware</th></tr><tr><td>C.parallelize(i)</td><td>Parallelize the i loop level for execution on a shared memory system.</td></tr><tr><td>C.vectorize(i,v)</td><td>Vectorize the loop level i by a vector size v.</td></tr><tr><td>C.gpu(i0,i1,i2,i3)</td><td>Mark the loop levels i0,i1,i2 and i3 to be executed on GPU. (i0,i1) are mapped to block IDs and (i2,i3) to thread IDs.</td></tr><tr><td>C.tile_gpu(i0,i1,t1,t2)</td><td>Tile the loops i0 and i1 by $t1 \times t2$ and map them to GPU.</td></tr><tr><td>C.distribute(i)</td><td>Parallelize the loop level i for execution on a distributed memory system.</td></tr></table>	We assume that C and P are computations, b is a buffer i and j are loop iterators		Commands for loop nest transformations		Command	Description	C.tile(i,j,t1,t2,i0,j0,i1,j1)	Tile the loop levels (i,j) of the computation C by $t1 \times t2$. The names of the new loop levels are (i0,j0,i1,j1) where i0 is the outermost loop level and j1 is the innermost.	C.interchange(i,j)	Interchange the i and j loop levels of C.	C.shift(i,s)	Loop shifting (shift the loop level i by s iterations).	C.split(i,s,i0,i1)	Split the loop level i by s. (i0,i1) are the new loop levels.	P.compute_at(C,j)	Compute the computation P in the loop nest of C at loop level j. This might introduce redundant computations.	C.unroll(i,v)	Unroll the loop level i by a factor v.	C.after(B,i)	Indicate that C should be ordered after B at the loop level i (they have the same order in all the loop levels above i).	C.inline()	Inline C in all of its consumers.	C.set_schedule()	Transform the iteration domain of C using an affine relation (a map to transform Layer I to II expressed in the ISL syntax).	Commands for mapping loop levels to hardware		C.parallelize(i)	Parallelize the i loop level for execution on a shared memory system.	C.vectorize(i,v)	Vectorize the loop level i by a vector size v.	C.gpu(i0,i1,i2,i3)	Mark the loop levels i0,i1,i2 and i3 to be executed on GPU. (i0,i1) are mapped to block IDs and (i2,i3) to thread IDs.	C.tile_gpu(i0,i1,t1,t2)	Tile the loops i0 and i1 by $t1 \times t2$ and map them to GPU.	C.distribute(i)	Parallelize the loop level i for execution on a distributed memory system.
We assume that C and P are computations, b is a buffer i and j are loop iterators																																									
Commands for loop nest transformations																																									
Command	Description																																								
C.tile(i,j,t1,t2,i0,j0,i1,j1)	Tile the loop levels (i,j) of the computation C by $t1 \times t2$. The names of the new loop levels are (i0,j0,i1,j1) where i0 is the outermost loop level and j1 is the innermost.																																								
C.interchange(i,j)	Interchange the i and j loop levels of C.																																								
C.shift(i,s)	Loop shifting (shift the loop level i by s iterations).																																								
C.split(i,s,i0,i1)	Split the loop level i by s. (i0,i1) are the new loop levels.																																								
P.compute_at(C,j)	Compute the computation P in the loop nest of C at loop level j. This might introduce redundant computations.																																								
C.unroll(i,v)	Unroll the loop level i by a factor v.																																								
C.after(B,i)	Indicate that C should be ordered after B at the loop level i (they have the same order in all the loop levels above i).																																								
C.inline()	Inline C in all of its consumers.																																								
C.set_schedule()	Transform the iteration domain of C using an affine relation (a map to transform Layer I to II expressed in the ISL syntax).																																								
Commands for mapping loop levels to hardware																																									
C.parallelize(i)	Parallelize the i loop level for execution on a shared memory system.																																								
C.vectorize(i,v)	Vectorize the loop level i by a vector size v.																																								
C.gpu(i0,i1,i2,i3)	Mark the loop levels i0,i1,i2 and i3 to be executed on GPU. (i0,i1) are mapped to block IDs and (i2,i3) to thread IDs.																																								
C.tile_gpu(i0,i1,t1,t2)	Tile the loops i0 and i1 by $t1 \times t2$ and map them to GPU.																																								
C.distribute(i)	Parallelize the loop level i for execution on a distributed memory system.																																								

Identifier	Page	Conf.	LaTeX source	Rendered	Image												
1804.10694v5_TAB_003	4	■ 0.726	<pre>\begin{tabular}[t]{t}{ l l } \hline \multicolumn{2}{ c } {High level commands for data manipulation} \\ \hline C.store_in (b, \{i, j\}) & Store the result of the computation \(\mathrm{C}(\mathrm{i}, \mathrm{j})\) in \(\mathrm{b}[\mathrm{i}, \mathrm{j}]\). \\ \hline C.cache_shared_at (P, i) & Cache (copy) the buffer of C in shared memory. Copying from global to shared GPU memory will be done at loop level i of the computation P. The amount of data to copy, the access functions, and synchronization are computed automatically. \\ \hline C.cache_local_at (P, i) & Similar to cache_shared_at but stores in local GPU memory. \\ \hline send(d, src, s, q, p) & Create a send operation. d: vector of iterators to represent the iteration domain of the send; src: source buffer; s: size; q: destination node; p: properties (synchronous, asynchronous, blocking, ...). \\ \hline receive (d, dst, s, q, p) & Create a receive operation. Arguments similar to send except q, which is the source node. \\ \hline \end{tabular}</pre>	(not rendered)	<table><tr><th colspan="2">High level commands for data manipulation</th></tr><tr><td>C.store_in(b, {i, j})</td><td>Store the result of the computation C(i,j) in b[i,j].</td></tr><tr><td>C.cache_shared_at(P, i)</td><td>Cache (copy) the buffer of C in shared memory. Copying from global to shared GPU memory will be done at loop level i of the computation P. The amount of data to copy, the access functions, and synchronization are computed automatically.</td></tr><tr><td>C.cache_local_at(P, i)</td><td>Similar to cache_shared_at but stores in local GPU memory.</td></tr><tr><td>send(d, src, s, q, p)</td><td>Create a send operation. d: vector of iterators to represent the iteration domain of the send; src: source buffer; s: size; q: destination node; p: properties (synchronous, asynchronous, blocking, ...).</td></tr><tr><td>receive(d, dst, s, q, p)</td><td>Create a receive operation. Arguments similar to send except q, which is the source node.</td></tr></table>	High level commands for data manipulation		C.store_in (b, {i, j})	Store the result of the computation C(i,j) in b[i,j].	C.cache_shared_at (P, i)	Cache (copy) the buffer of C in shared memory. Copying from global to shared GPU memory will be done at loop level i of the computation P. The amount of data to copy, the access functions, and synchronization are computed automatically.	C.cache_local_at (P, i)	Similar to cache_shared_at but stores in local GPU memory.	send (d, src, s, q, p)	Create a send operation. d: vector of iterators to represent the iteration domain of the send; src: source buffer; s: size; q: destination node; p: properties (synchronous, asynchronous, blocking, ...).	receive (d, dst, s, q, p)	Create a receive operation. Arguments similar to send except q, which is the source node.
High level commands for data manipulation																	
C.store_in (b, {i, j})	Store the result of the computation C(i,j) in b[i,j].																
C.cache_shared_at (P, i)	Cache (copy) the buffer of C in shared memory. Copying from global to shared GPU memory will be done at loop level i of the computation P. The amount of data to copy, the access functions, and synchronization are computed automatically.																
C.cache_local_at (P, i)	Similar to cache_shared_at but stores in local GPU memory.																
send (d, src, s, q, p)	Create a send operation. d: vector of iterators to represent the iteration domain of the send; src: source buffer; s: size; q: destination node; p: properties (synchronous, asynchronous, blocking, ...).																
receive (d, dst, s, q, p)	Create a receive operation. Arguments similar to send except q, which is the source node.																

Identifier	Page	Conf.	LaTeX source	Rendered	Image																								
1804.10694v5_TAB_004	4	■0.630	<pre>\begin{tabular}[t]{l l}\hline \multicolumn{2}{ c } {Low level commands for data manipulation} \\ \hline Buffer b(sizes, type) & Declare a buffer (sizes: a vector of dimension sizes). \\ \hline b.allocate_at(p, i) & Return an operation that allocates b at the loop i of p . An operation can be scheduled like any computation. \\ \hline C.buffer() & Return the buffer associated to the computation C. \\ \hline b.set_size(sizes) & Set the size of a buffer. sizes: a vector of dimension sizes. \\ \hline b.tag_gpu_global() & Tag buffer to be stored in global GPU memory. \\ \hline b.tag_gpu_shared() & Tag buffer to be stored in shared GPU memory. \\ \hline b.tag_gpu_local() & Tag buffer to be stored in local GPU memory. \\ \hline b.tag_gpu_constant() & Tag buffer to be stored in constant GPU memory. \\ \hline C.host_to_device() & Return an operation that copies C.buffer() from host to device. \\ \hline C.device_to_host() & Return an operation that copies C.buffer() from device to host. \\ \hline copy_at(p, i, bs, bd) & Return an operation that copies the buffer bs to the buffer bd at the loop i of p . Used for copies between global, shared and local. \\ \hline \end{tabular}</pre>	(not rendered)	<table><tr><th colspan="2">Low level commands for data manipulation</th></tr><tr><td>Buffer b(sizes, type)</td><td>Declare a buffer (sizes: a vector of dimension sizes).</td></tr><tr><td>b.allocate_at(p, i)</td><td>Return an operation that allocates b at the loop i of p. An operation can be scheduled like any computation.</td></tr><tr><td>C.buffer()</td><td>Return the buffer associated to the computation C.</td></tr><tr><td>b.set_size(sizes)</td><td>Set the size of a buffer. sizes: a vector of dimension sizes.</td></tr><tr><td>b.tag_gpu_global()</td><td>Tag buffer to be stored in global GPU memory.</td></tr><tr><td>b.tag_gpu_shared()</td><td>Tag buffer to be stored in shared GPU memory.</td></tr><tr><td>b.tag_gpu_local()</td><td>Tag buffer to be stored in local GPU memory.</td></tr><tr><td>b.tag_gpu_constant()</td><td>Tag buffer to be stored in constant GPU memory.</td></tr><tr><td>C.host_to_device()</td><td>Return an operation that copies C.buffer() from host to device.</td></tr><tr><td>C.device_to_host()</td><td>Return an operation that copies C.buffer() from device to host.</td></tr><tr><td>copy_at(p, i, bs, bd)</td><td>Return an operation that copies the buffer bs to the buffer bd at the loop i of p. Used for copies between global, shared and local.</td></tr></table>	Low level commands for data manipulation		Buffer b(sizes, type)	Declare a buffer (sizes: a vector of dimension sizes).	b.allocate_at(p, i)	Return an operation that allocates b at the loop i of p. An operation can be scheduled like any computation.	C.buffer()	Return the buffer associated to the computation C.	b.set_size(sizes)	Set the size of a buffer. sizes: a vector of dimension sizes.	b.tag_gpu_global()	Tag buffer to be stored in global GPU memory.	b.tag_gpu_shared()	Tag buffer to be stored in shared GPU memory.	b.tag_gpu_local()	Tag buffer to be stored in local GPU memory.	b.tag_gpu_constant()	Tag buffer to be stored in constant GPU memory.	C.host_to_device()	Return an operation that copies C.buffer() from host to device.	C.device_to_host()	Return an operation that copies C.buffer() from device to host.	copy_at(p, i, bs, bd)	Return an operation that copies the buffer bs to the buffer bd at the loop i of p. Used for copies between global, shared and local.
Low level commands for data manipulation																													
Buffer b(sizes, type)	Declare a buffer (sizes: a vector of dimension sizes).																												
b.allocate_at(p, i)	Return an operation that allocates b at the loop i of p. An operation can be scheduled like any computation.																												
C.buffer()	Return the buffer associated to the computation C.																												
b.set_size(sizes)	Set the size of a buffer. sizes: a vector of dimension sizes.																												
b.tag_gpu_global()	Tag buffer to be stored in global GPU memory.																												
b.tag_gpu_shared()	Tag buffer to be stored in shared GPU memory.																												
b.tag_gpu_local()	Tag buffer to be stored in local GPU memory.																												
b.tag_gpu_constant()	Tag buffer to be stored in constant GPU memory.																												
C.host_to_device()	Return an operation that copies C.buffer() from host to device.																												
C.device_to_host()	Return an operation that copies C.buffer() from device to host.																												
copy_at(p, i, bs, bd)	Return an operation that copies the buffer bs to the buffer bd at the loop i of p. Used for copies between global, shared and local.																												
1804.10694v5_TAB_005	4	■0.999	<pre>\begin{tabular}[t]{l l}\hline \multicolumn{2}{ c }{ Commands for synchronization} \\ \hline barrier_at(p, i) & Create a barrier at the loop p of i. \\ \hline \end{tabular}</pre>	(not rendered)	<table><tr><th colspan="2">Commands for synchronization</th></tr><tr><td>barrier_at(p, i)</td><td>Create a barrier at the loop p of i.</td></tr></table>	Commands for synchronization		barrier_at(p, i)	Create a barrier at the loop p of i.																				
Commands for synchronization																													
barrier_at(p, i)	Create a barrier at the loop p of i.																												

Identifier	Page	Conf.	LaTeX source	Rendered	Image																																																																																					
1804.10694v5_TAB_006	11	■ 1.000	<pre>\begin{tabular}[t]{t}{ l l l l l l l l l } \hline Architectures & \multicolumn{8}{ l }{Frameworks} \\ \hline & & edge Detector & cvtColor & Conv2D & warp Affine & gaussian & nb & ticket \#2373 \\ \hline & & & & & & & & \\ \hline Single-node multicore & Tiramisu & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ & Halide & - & 1 & 1 & 1 & 1 & 3.77 & - \\ & PENCIL & 2.43 & 2.39 & 11.82 & 10.2 & 5.82 & 1 & 1 \\ \hline GPU & Tiramisu & 1.05 & 1 & 1 & 1 & 1 & 1 & 1 \\ & Halide & - & 1 & 1.3 & 1 & 1.3 & 1.7 & - \\ & PENCIL & 1 & 1 & 1.33 & 1 & 1.2 & 1.02 & 1 \\ \hline Distributed (16 Nodes) & Tiramisu & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ & Dist-Halide & - & 1.31 & 3.25 & 2.54 & 1.57 & 1.45 & - \\ \hline \end{tabular}</pre>	(not rendered)	<table><tr><th>Architectures</th><th>Frameworks</th><th colspan="7">Benchmarks</th></tr><tr><th></th><th></th><th>edge Detector</th><th>cvtColor</th><th>Conv2D</th><th>warp Affine</th><th>gaussian</th><th>nb</th><th>ticket #2373</th></tr><tr><td rowspan="3">Single-node multicore</td><td>Tiramisu</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>Halide</td><td>-</td><td>1</td><td>1</td><td>1</td><td>1</td><td>3.77</td><td>-</td></tr><tr><td>PENCIL</td><td>2.43</td><td>2.39</td><td>11.82</td><td>10.2</td><td>5.82</td><td>1</td><td>1</td></tr><tr><td rowspan="3">GPU</td><td>Tiramisu</td><td>1.05</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>Halide</td><td>-</td><td>1</td><td>1.3</td><td>1</td><td>1.3</td><td>1.7</td><td>-</td></tr><tr><td>PENCIL</td><td>1</td><td>1</td><td>1.33</td><td>1</td><td>1.2</td><td>1.02</td><td>1</td></tr><tr><td rowspan="2">Distributed (16 Nodes)</td><td>Tiramisu</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>Dist-Halide</td><td>-</td><td>1.31</td><td>3.25</td><td>2.54</td><td>1.57</td><td>1.45</td><td>-</td></tr></table>	Architectures	Frameworks	Benchmarks									edge Detector	cvtColor	Conv2D	warp Affine	gaussian	nb	ticket #2373	Single-node multicore	Tiramisu	1	1	1	1	1	1	1	Halide	-	1	1	1	1	3.77	-	PENCIL	2.43	2.39	11.82	10.2	5.82	1	1	GPU	Tiramisu	1.05	1	1	1	1	1	1	Halide	-	1	1.3	1	1.3	1.7	-	PENCIL	1	1	1.33	1	1.2	1.02	1	Distributed (16 Nodes)	Tiramisu	1	1	1	1	1	1	1	Dist-Halide	-	1.31	3.25	2.54	1.57	1.45	-
Architectures	Frameworks	Benchmarks																																																																																								
		edge Detector	cvtColor	Conv2D	warp Affine	gaussian	nb	ticket #2373																																																																																		
Single-node multicore	Tiramisu	1	1	1	1	1	1	1																																																																																		
	Halide	-	1	1	1	1	3.77	-																																																																																		
	PENCIL	2.43	2.39	11.82	10.2	5.82	1	1																																																																																		
GPU	Tiramisu	1.05	1	1	1	1	1	1																																																																																		
	Halide	-	1	1.3	1	1.3	1.7	-																																																																																		
	PENCIL	1	1	1.33	1	1.2	1.02	1																																																																																		
Distributed (16 Nodes)	Tiramisu	1	1	1	1	1	1	1																																																																																		
	Dist-Halide	-	1.31	3.25	2.54	1.57	1.45	-																																																																																		